



Universidade Federal
de São João del-Rei

Programa de Pós-Graduação em Ciência da Computação
Projeto e Análise de Algoritmos

Documentação Trabalho Prático 2

Lívia Dâmaso, Lucas Bergami, Sávio Francisco

1 Introdução

O presente trabalho tem como objetivo resolver o problema que consiste em determinar a maior pontuação possível obtida a partir da seleção de elementos de uma sequência de inteiros sob restrições específicas de escolha.

No contexto do problema, um jogador recebe uma sequência contendo N valores inteiros e pode realizar diversas jogadas. Em cada jogada, um elemento da sequência é selecionado para pontuação e, simultaneamente, seus elementos vizinhos imediatos tornam-se indisponíveis para futuras escolhas. O objetivo consiste em determinar a combinação de escolhas que maximize a soma total dos valores obtidos ao final do processo.

Do ponto de vista computacional, o problema pode ser interpretado como um problema clássico de otimização combinatória baseado em decisões locais com impacto global. Embora a escolha gulosa do maior valor disponível pareça intuitiva em um primeiro momento, ela não garante a obtenção da solução ótima, tornando necessária a utilização de estratégias capazes de considerar diferentes combinações possíveis de forma eficiente.

Este problema possui aplicações em diversos cenários computacionais relacionados à tomada de decisão sob restrições, como alocação de recursos, planejamento de tarefas independentes, seleção de atividades conflitantes e problemas de maximização em estruturas lineares.

Para resolver o problema proposto, foram implementadas duas estratégias distintas. A primeira utiliza Programação Dinâmica na abordagem *Bottom-Up*, explorando a reutilização de subproblemas para construir iterativamente a solução ótima. A segunda utiliza Programação Dinâmica na abordagem *Top-Down* com memoização, empregando recursão aliada ao armazenamento intermediário dos resultados para evitar recomputações.

Além da implementação das estratégias, também foi realizada uma análise de desempenho baseada em medições experimentais de tempo de execução, permitindo comparar o comportamento das abordagens adotadas e verificar sua escalabilidade para diferentes tamanhos de entrada.

Nas próximas seções serão apresentados a arquitetura geral da solução desenvolvida, os algoritmos relacionados ao problema, a descrição detalhada das abordagens implementadas, a análise matemática da complexidade computacional e os resultados obtidos nos testes experimentais.

2 Arquitetura da solução

Após compreender o problema e definir as estratégias que seriam utilizadas para sua resolução, foi necessário estabelecer o fluxo de execução do programa. A arquitetura proposta organiza a solução em etapas responsáveis pela leitura dos dados de entrada, execução da estratégia selecionada, geração do resultado e coleta dos tempos de execução.

De forma geral, o programa inicia recebendo os parâmetros via linha de comando, realiza a leitura do arquivo de entrada, executa o algoritmo correspondente à estratégia escolhida e, por fim, grava a resposta em um arquivo de saída enquanto exibe informações de desempenho no terminal.

2.1 Receber dados da linha de comando

A execução do programa ocorre por meio da linha de comando, permitindo que o usuário escolha dinamicamente qual estratégia será utilizada na resolução do problema.

O executável recebe dois argumentos:

- Estratégia utilizada na resolução (D ou A);
- Arquivo contendo os dados de entrada.

A chamada do programa segue o seguinte formato:

```
./tp2 <estrategia> entrada.txt
```

Onde:

- D: estratégia baseada em Programação Dinâmica Bottom-Up;
- A: estratégia baseada em Programação Dinâmica Top-Down com memoização.

2.2 Leitura do arquivo de entrada

Após interpretar os argumentos fornecidos, o programa realiza a abertura do arquivo de entrada e carrega os dados necessários para processamento.

O formato utilizado consiste em:

- Primeira linha contendo o valor inteiro N , correspondente ao tamanho da sequência;
- Segunda linha contendo os N valores inteiros da sequência.

Exemplo:

```
9
1 2 1 3 2 2 2 2 3
```

Os valores lidos são armazenados dinamicamente em memória para permitir o processamento eficiente independentemente do tamanho da entrada.

2.3 Determinação da pontuação máxima

Com os dados carregados, o programa executa a estratégia escolhida.

Na abordagem baseada em Programação Dinâmica Bottom-Up, a solução é construída iterativamente a partir das soluções ótimas dos prefixos do vetor, evitando recomputações e obtendo desempenho linear.

Na abordagem Top-Down com memoização, o problema é resolvido recursivamente, armazenando resultados intermediários já calculados para impedir chamadas redundantes.

Em ambas as estratégias, o objetivo é determinar o maior valor acumulado possível respeitando a restrição de que elementos adjacentes não podem ser escolhidos simultaneamente.

2.4 Salvar resultados e medir desempenho

Ao final da execução, o valor correspondente à pontuação máxima obtida é armazenado no arquivo `saida.txt`.

Além disso, são coletadas métricas de desempenho utilizando as funções `gettimeofday()` e `getrusage()`, permitindo medir os tempos de usuário e sistema consumidos durante a execução.

Essas informações possibilitam comparar experimentalmente as estratégias implementadas e analisar seu comportamento conforme o tamanho da entrada aumenta.

3 Algoritmos similares

Antes da definição da solução final implementada neste trabalho, foram analisadas diferentes abordagens clássicas utilizadas em problemas de otimização sob restrições. O objetivo dessa etapa foi compreender alternativas possíveis para resolução do problema e justificar a escolha dos algoritmos adotados.

O problema estudado apresenta características típicas de otimização combinatória, nas quais decisões tomadas localmente afetam as possibilidades futuras de escolha. Dessa forma, diferentes estratégias podem ser consideradas para sua resolução.

3.1 Força Bruta

Uma abordagem direta consiste em explorar todas as combinações possíveis de elementos da sequência, verificando quais respeitam a restrição de não selecionar elementos adjacentes.

Essa estratégia pode ser implementada utilizando recursão ou retrocesso (*backtracking*), onde cada posição da sequência gera duas possibilidades:

- selecionar o elemento atual;
- ignorar o elemento atual.

Ao selecionar um elemento, o próximo elemento torna-se indisponível, fazendo com que a busca continue a partir da posição subsequente permitida.

Embora essa abordagem sempre encontre a solução ótima, seu custo computacional cresce exponencialmente com o tamanho da entrada.

Sua complexidade temporal pode ser aproximada por:

$$T(n) = T(n - 1) + T(n - 2)$$

que resulta em $O(2^n)$. Essa característica torna sua utilização inviável para valores elevados de N .

3.2 Algoritmos Gulosos

Outra alternativa considerada foi o uso de estratégias gulosas (*greedy*)[1], nas quais as decisões são tomadas com base apenas na melhor escolha local disponível.

Uma possível heurística seria selecionar sempre o maior valor disponível e remover seus elementos adjacentes.

Apesar da simplicidade e do baixo custo computacional, essa abordagem não garante a obtenção da solução ótima.

Considere o exemplo:

$$[1, 2, 5, 4]$$

Uma estratégia gulosa poderia selecionar inicialmente o valor 5 e em seguida o valor 1, obtendo resultado final igual a 6. Entretanto, a solução ótima corresponde à seleção dos valores 4 e 2, produzindo soma total igual a 6.

Dessa forma, algoritmos gulosos foram descartados por não garantirem corretude.

3.3 Programação Dinâmica

Problemas que apresentam sobreposição de subproblemas e subestrutura ótima podem ser resolvidos eficientemente utilizando Programação Dinâmica[1].

Nesse tipo de abordagem, soluções previamente calculadas são reutilizadas para evitar re-computações.

Para cada posição da sequência, duas possibilidades são avaliadas:

- selecionar o elemento atual e somar à melhor solução encontrada anteriormente;
- ignorar o elemento atual e manter a melhor solução já conhecida.

Essa característica reduz significativamente o custo computacional, permitindo resolver entradas muito maiores.

A recorrência utilizada é definida como:

$$DP(i) = \max(DP(i - 1), DP(i - 2) + a_i)$$

onde:

- $DP(i - 1)$ representa ignorar o elemento atual;
- $DP(i - 2) + a_i$ representa selecionar o elemento atual.

Essa abordagem apresenta complexidade temporal linear $O(n)$, tornando-se adequada para os limites estabelecidos no problema.

Após a análise das alternativas apresentadas, optou-se pela utilização de Programação Dinâmica como solução principal do trabalho, além da implementação de uma variação baseada em memoização para fins de comparação experimental.

4 Descrição detalhada das soluções implementadas

Após a análise das alternativas possíveis para resolução do problema, foram implementadas duas abordagens baseadas em Programação Dinâmica. Embora ambas utilizem o mesmo princípio de reutilização de subproblemas, diferem na forma como os estados são explorados e armazenados durante a execução.

As duas estratégias implementadas foram:

- Programação Dinâmica Bottom-Up (Estratégia D);
- Programação Dinâmica Top-Down com Memoização (Estratégia A).

Em ambas as abordagens, o objetivo consiste em determinar a maior pontuação possível ao selecionar elementos da sequência respeitando a restrição de que elementos adjacentes não podem ser escolhidos simultaneamente.

4.1 Observação sobre a modelagem do problema

Embora o enunciado descreva que, ao selecionar um elemento, seus vizinhos são removidos da sequência e os elementos restantes passam a ocupar novas posições, essa atualização não altera a estrutura do problema.

Isso ocorre porque um elemento removido nunca poderá voltar a ser selecionado posteriormente, independentemente do deslocamento dos elementos remanescentes.

Dessa forma, o problema pode ser reinterpretado como:

Selecionar elementos da sequência original de modo que dois elementos adjacentes nunca sejam escolhidos simultaneamente.

Essa observação permite transformar o problema em uma formulação clássica de otimização sobre sequências.

4.2 Estratégia D — Programação Dinâmica Bottom-Up

A primeira solução implementada utiliza Programação Dinâmica na abordagem Bottom-Up.

Nessa técnica, o algoritmo constrói iterativamente a resposta final utilizando soluções menores previamente calculadas.

Foi definido o vetor $DP[i]$, que representa a maior pontuação possível considerando apenas os elementos entre as posições $0 \dots i$. Para calcular cada posição, são avaliadas duas possibilidades:

Não selecionar o elemento atual

Caso o elemento atual não seja escolhido, a melhor solução permanece igual à encontrada anteriormente:

$$DP[i - 1]$$

Selecionar o elemento atual

Caso o elemento atual seja escolhido, o elemento anterior não pode ter sido utilizado. Nesse caso:

$$DP[i] = DP[i - 2] + a[i]$$

Onde:

- $a[i]$ representa o valor do elemento armazenado na posição i da sequência original;
- $DP[i - 1]$ representa a melhor solução sem selecionar o elemento atual;
- $DP[i - 2] + a[i]$ representa a seleção do elemento atual somada à melhor solução compatível anterior.

Portanto, a recorrência utilizada é:

$$DP[i] = \max(DP[i - 1], DP[i - 2] + a[i])$$

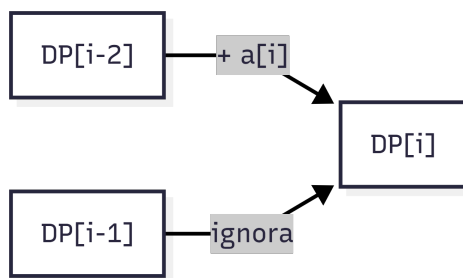


Figura 1: Diagrama de transição de estados

As condições iniciais utilizadas foram:

$$DP[0] = a[0]$$

$$DP[1] = \max(a[0], a[1])$$

A solução é construída da esquerda para a direita até atingir o último elemento do vetor. Exemplo:

[1, 2, 1, 3, 2]

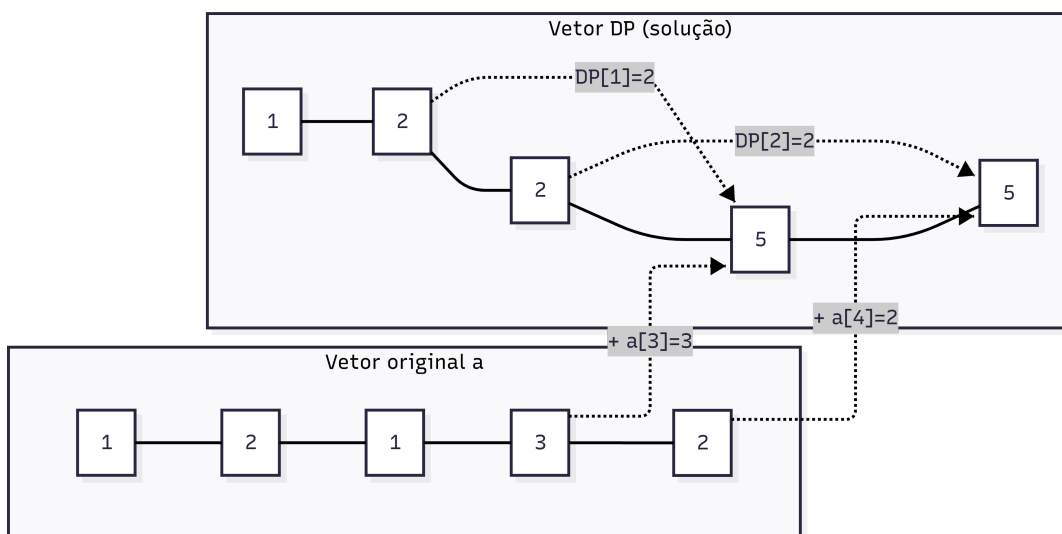


Figura 2: Ilustração da solução

O processo de construção da tabela ocorre da esquerda para a direita, conforme detalhado a seguir:

i	$a[i]$	$DP[i]$	Cálculo/Justificativa
0	1	1	Caso base: $DP[0] = a[0]$
1	2	2	Caso base: $\max(a[0], a[1]) = \max(1, 2)$
2	1	2	$\max(DP[1], DP[0] + a[2]) = \max(2, 1 + 1)$
3	3	5	$\max(DP[2], DP[1] + a[3]) = \max(2, 2 + 3)$
4	2	5	$\max(DP[3], DP[2] + a[4]) = \max(5, 2 + 2)$

Tabela 1: Evolução do vetor DP passo a passo.

O resultado final do problema para a sequência dada está contido na última posição preenchida ($DP[4] = 5$).

A principal vantagem dessa abordagem é sua simplicidade de implementação e desempenho linear.

Complexidade:

$$Tempo = O(n)$$

$$Espao = O(n)$$

4.3 Estratégia A — Programação Dinâmica Top-Down com memoização

A segunda estratégia implementada utiliza uma abordagem recursiva baseada em Programação Dinâmica Top-Down. Ao contrário da versão Bottom-Up, nessa abordagem a solução é construída sob demanda.

Foi definida uma função recursiva $F(i)$ que representa a maior pontuação possível iniciando a análise a partir da posição i . Para cada chamada recursiva existem duas possibilidades:

Selecionar elemento atual

Ao selecionar o elemento da posição atual:

$$a[i]$$

o próximo elemento não poderá ser utilizado. Logo:

$$a[i] + F(i + 2)$$

Ignorar elemento atual

Nesse caso:

$$F(i + 1)$$

A recorrência utilizada torna-se:

$$F(i) = \max(a[i] + F(i + 2), F(i + 1))$$

Os casos base são definidos por:

$$F(i) = 0$$

quando $i \geq n$.

Para evitar recomputações, foi utilizado um vetor auxiliar denominado *memo*. Sempre que um estado já tiver sido calculado anteriormente, seu valor é reutilizado imediatamente. Exemplo:

$$F(0)$$

gera:

$$\max(a[0] + F(2), F(1))$$

e cada subproblema é resolvido apenas uma vez.

Sem memoização, a árvore de chamadas apresentaria crescimento exponencial. Com memoização, o número de estados distintos calculados é limitado a N .

Complexidade:

$$Tempo = O(n)$$

$$Espao = O(n)$$

4.4 Comparação entre as estratégias

Embora ambas apresentem a mesma complexidade assintótica, o comportamento interno das soluções é diferente.

A abordagem Bottom-Up realiza os cálculos iterativamente utilizando um vetor de estados previamente definido. Nesse modelo, as soluções dos subproblemas são construídas progressivamente até atingir a resposta final.

Já a abordagem Top-Down utiliza recursão para decompor o problema em subproblemas menores e resolve apenas os estados que forem necessários durante a execução. Para evitar recomputações, utiliza um mecanismo de memoização que armazena resultados previamente calculados.

Apesar de ambas produzirem os mesmos resultados e apresentarem complexidade temporal linear quando implementadas com Programação Dinâmica, a abordagem recursiva possui algumas desvantagens práticas.

Cada chamada recursiva adiciona um novo quadro à pilha de execução (*call stack*), aumentando o consumo de memória durante a execução do programa. Em entradas muito grandes, isso pode causar estouro de pilha (*stack overflow*), interrompendo a execução.

Além disso, chamadas recursivas introduzem um custo adicional associado ao empilhamento e desempilhamento das funções, tornando a execução ligeiramente mais lenta em comparação com abordagens iterativas equivalentes.

No contexto deste trabalho, como o valor máximo de entrada pode atingir 10^5 elementos, a solução Bottom-Up apresenta maior robustez prática por evitar limitações relacionadas à profundidade da recursão.

Dessa forma, embora a abordagem Top-Down ofereça uma implementação conceitualmente mais próxima da definição recursiva do problema e facilite o entendimento da formulação matemática, a abordagem Bottom-Up tende a apresentar melhor eficiência prática e maior segurança para entradas de grande escala.

5 Estruturas de dados utilizadas

A implementação desenvolvida utiliza uma quantidade reduzida de estruturas de dados, uma vez que o problema possui natureza sequencial e pode ser resolvido armazenando apenas estados intermediários.

As estruturas utilizadas foram escolhidas com o objetivo de manter simplicidade de implementação e eficiência computacional.

5.1 Vetor de entrada

A sequência recebida no arquivo de entrada é armazenada em um vetor dinâmico de inteiros:

$$a[0], a[1], \dots, a[n - 1]$$

Cada posição do vetor representa o valor associado a uma possível escolha do jogador.

A utilização de alocação dinâmica permite que o programa suporte diferentes tamanhos de entrada sem desperdício de memória.

A leitura é realizada por meio de:

```
int *a = malloc(n * sizeof(int));
```

onde:

- n representa a quantidade de elementos da sequência;
- cada posição contém um valor inteiro da entrada.

Essa estrutura é utilizada por ambas as estratégias implementadas.

5.2 Vetor de Programação Dinâmica

Na estratégia Bottom-Up foi utilizado um vetor auxiliar denominado:

$$DP[0], DP[1], \dots, DP[n - 1]$$

Esse vetor armazena soluções ótimas parciais. Cada posição $DP[i]$ representa a maior pontuação possível considerando os elementos entre as posições $0 \dots i$. Seu preenchimento ocorre sequencialmente até atingir a solução final.

A alocação é realizada dinamicamente:

```
long long *dp = malloc(n * sizeof(long long));
```

Foi utilizado o tipo `long long` para evitar estouro de inteiro em entradas grandes, já que o valor acumulado pode ultrapassar o limite de um inteiro convencional.

5.3 Vetor de Memoização

Na estratégia Top-Down foi utilizado um vetor global de memoização:

$$memo[0], memo[1], \dots, memo[n - 1]$$

Seu objetivo é armazenar resultados previamente calculados durante a execução recursiva.

Cada posição $memo[i]$ representa a melhor solução possível iniciando na posição i . Antes da execução, o vetor é inicializado com -1, indicando estados ainda não calculados. Sempre que um estado já tiver sido processado anteriormente, seu valor é reutilizado imediatamente, eliminando chamadas recursivas redundantes. Essa técnica reduz significativamente o número de operações realizadas e transforma uma solução originalmente exponencial em uma solução linear.

6 Análise detalhada da complexidade

Para avaliar a eficiência das soluções implementadas, foi realizada uma análise considerando o crescimento do tempo de execução e do consumo de memória em função do tamanho da entrada.

O tamanho da entrada é definido por n que representa a quantidade de elementos da sequência.

6.1 Complexidade da solução Bottom-Up

Na abordagem Bottom-Up existe apenas um laço principal responsável pelo preenchimento do vetor de Programação Dinâmica.

O algoritmo executa:

$$n - 2$$

iterações.

Em cada iteração são realizadas operações de custo constante:

- acesso ao vetor;
- soma;
- comparação;
- atribuição.

Portanto:

$$T(n) = c \cdot n$$

Como c representa uma constante, pode-se desprezá-la, logo:

$$T(n) = O(n)$$

Assim, o tempo cresce linearmente com o tamanho da entrada.

Em relação ao espaço utilizado:

- vetor de entrada: $O(n)$;
- vetor DP: $O(n)$.

Logo:

$$S(n) = O(n)$$

6.2 Complexidade da solução Top-Down com Memoização

Na abordagem Top-Down cada estado do problema é identificado pela posição atual do vetor. Sem memoização, cada chamada geraria duas novas chamadas:

$$T(n) = T(n-1) + T(n-2)$$

resultando em crescimento exponencial $O(2^n)$. Entretanto, utilizando memoização, cada posição é calculada no máximo uma única vez. Como existem apenas n estados distintos:

$$T(n) = O(n)$$

Apesar da mesma complexidade temporal assintótica da versão Bottom-Up, existe um custo adicional relacionado à recursão: cada chamada adiciona um quadro na pilha de execução. No pior caso:

$$P(n) = O(n)$$

onde P representa a profundidade máxima da pilha.

O espaço total utilizado torna-se:

$$S(n) = O(n)$$

devido à combinação entre:

- vetor de entrada;
- vetor de memoização;
- pilha de chamadas recursivas.

6.3 Comparação entre as soluções

A Tabela a seguir resume as características observadas.

Estratégia	Tempo	Espaço
Bottom-Up	$O(n)$	$O(n)$
Top-Down	$O(n)$	$O(n)$

Embora ambas apresentem crescimento linear, a abordagem Bottom-Up tende a apresentar desempenho prático superior devido à ausência de chamadas recursivas e menor sobrecarga de execução.

7 Testes

Para validar empiricamente o comportamento da solução frente ao crescimento do volume de dados de entrada, foi realizada uma bateria de testes isolando o impacto do parâmetro N . O ambiente de testes foi automatizado por um *script*, responsável por gerar vetores de tamanhos variando linearmente no intervalo $[100, 100.000]$. Para garantir a consistência estatística e mitigar oscilações espúrias causadas pelo sistema operacional, cada configuração de tamanho foi executada por 30 repetições, coletando-se o tempo real e o tempo de CPU consumido pelo processo.

Os dados resultantes foram consolidados e analisados estatisticamente. A Figura 3 apresenta o tempo médio de CPU em função do tamanho da entrada (N), acompanhado de barras de erro que representam o desvio padrão de cada amostra.

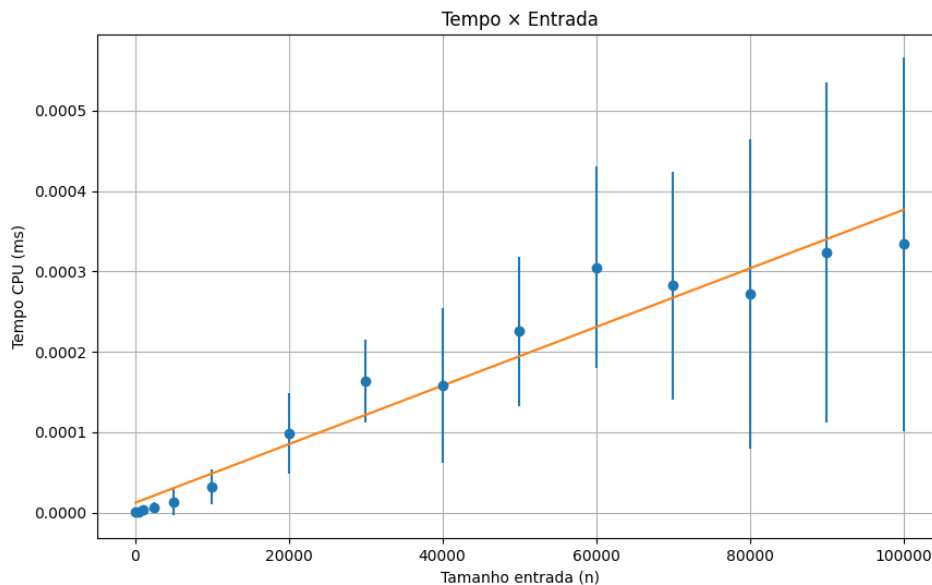


Figura 3: Tempo de CPU em função do tamanho da entrada N com regressão linear.

A partir do mapeamento dos pontos experimentais, aplicou-se o método dos mínimos quadrados para ajustar uma reta de regressão linear do tipo $f(x) = ax + b$. O coeficiente angular obtido ($a \approx \text{Inclinacao}$) reflete o custo computacional marginal por elemento inserido na entrada, enquanto o intercepto ($b \approx \text{Intercepto}$) representa o *overhead* constante inicial de inicialização do programa.

Inclinação: $3.6460431433658106e - 09$

Intercepto: $1.1997596347389363e - 05$

A forte aderência dos pontos experimentais à reta ajustada confirma de maneira empírica que a complexidade de tempo do algoritmo cresce de forma estritamente linear $O(N)$ em relação ao tamanho da entrada.

8 Conclusão

Este trabalho tratou de um problema de otimização combinatória baseado na seleção de elementos de uma sequência de inteiros sob a restrição de que elementos adjacentes não podem ser escolhidos simultaneamente. Para isso, foram estudadas diferentes abordagens de resolução, incluindo força bruta, algoritmos gulosos e Programação Dinâmica. A análise dessas alternativas demonstrou que estratégias de força bruta apresentam crescimento exponencial, tornando-se inviáveis para entradas de grande porte. Da mesma forma, abordagens gulosas não asseguram a solução ótima, o que as torna inadequadas quando se exige exatidão no resultado.

Diante dessas limitações, foram implementadas duas soluções baseadas em Programação Dinâmica: uma abordagem Bottom-Up e uma abordagem Top-Down com memoização. Ambas

exploram a propriedade de subestrutura ótima do problema e reaproveitam resultados intermediários para evitar recomputações desnecessárias.

A análise teórica mostrou que as duas estratégias apresentam complexidade temporal linear $O(n)$ e consumo de memória $O(n)$, representando uma melhora significativa em relação às abordagens exponenciais. Os experimentos realizados confirmam essa análise, evidenciando que o tempo de execução cresce de forma linear com o tamanho da entrada e permanece adequado mesmo para os maiores valores previstos. Embora as duas implementações produzam a mesma solução ótima, observou-se que a abordagem Bottom-Up apresenta vantagens práticas importantes. Por utilizar uma implementação iterativa, ela elimina a sobrecarga associada às chamadas recursivas e evita problemas relacionados à profundidade da pilha de execução, tornando-se mais robusta para entradas de grande escala.

Já a abordagem Top-down oferece formulação mais próxima da definição matemática recursiva do problema, facilitando o entendimento conceitual da solução. Seu uso, no entanto, implica custos adicionais de gerenciamento da pilha e possíveis limitações para valores muito altos de n .

Dessa forma, a Programação Dinâmica mostrou-se uma solução eficiente e adequada para o problema estudado. Os resultados obtidos demonstram que ambas as estratégias implementadas são capazes de encontrar a solução ótima dentro dos limites estabelecidos, sendo a abordagem Bottom-Up a alternativa mais indicada quando se busca maior eficiência prática e robustez computacional.

Referências

- [1] Thomas H. Cormen. *Algoritmos: Teoria e prática*. LTC, 2012.